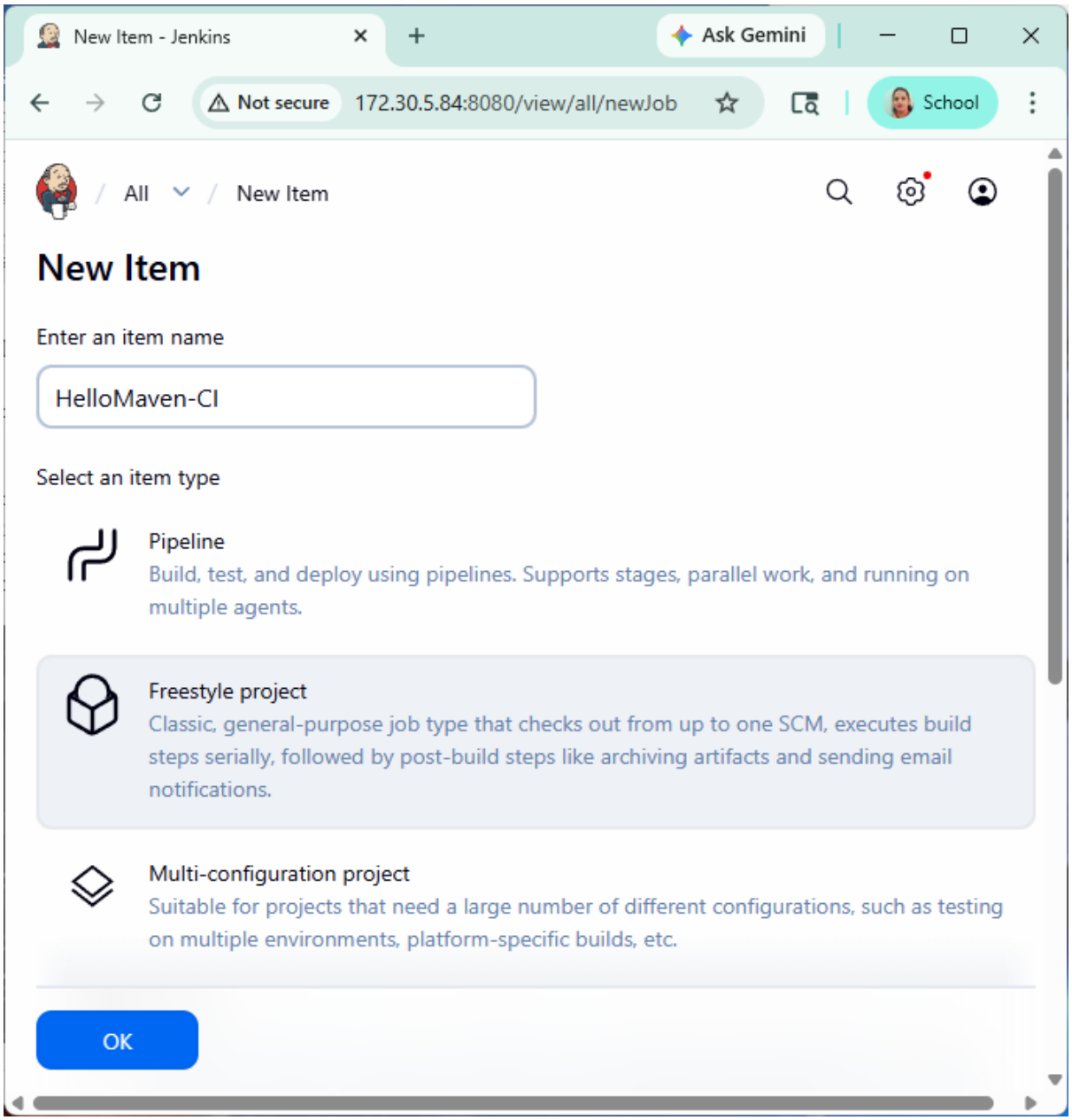
Experiment 8 - Jenkins and Ansible

1. Create a New Jenkins Job (Freestyle Project)

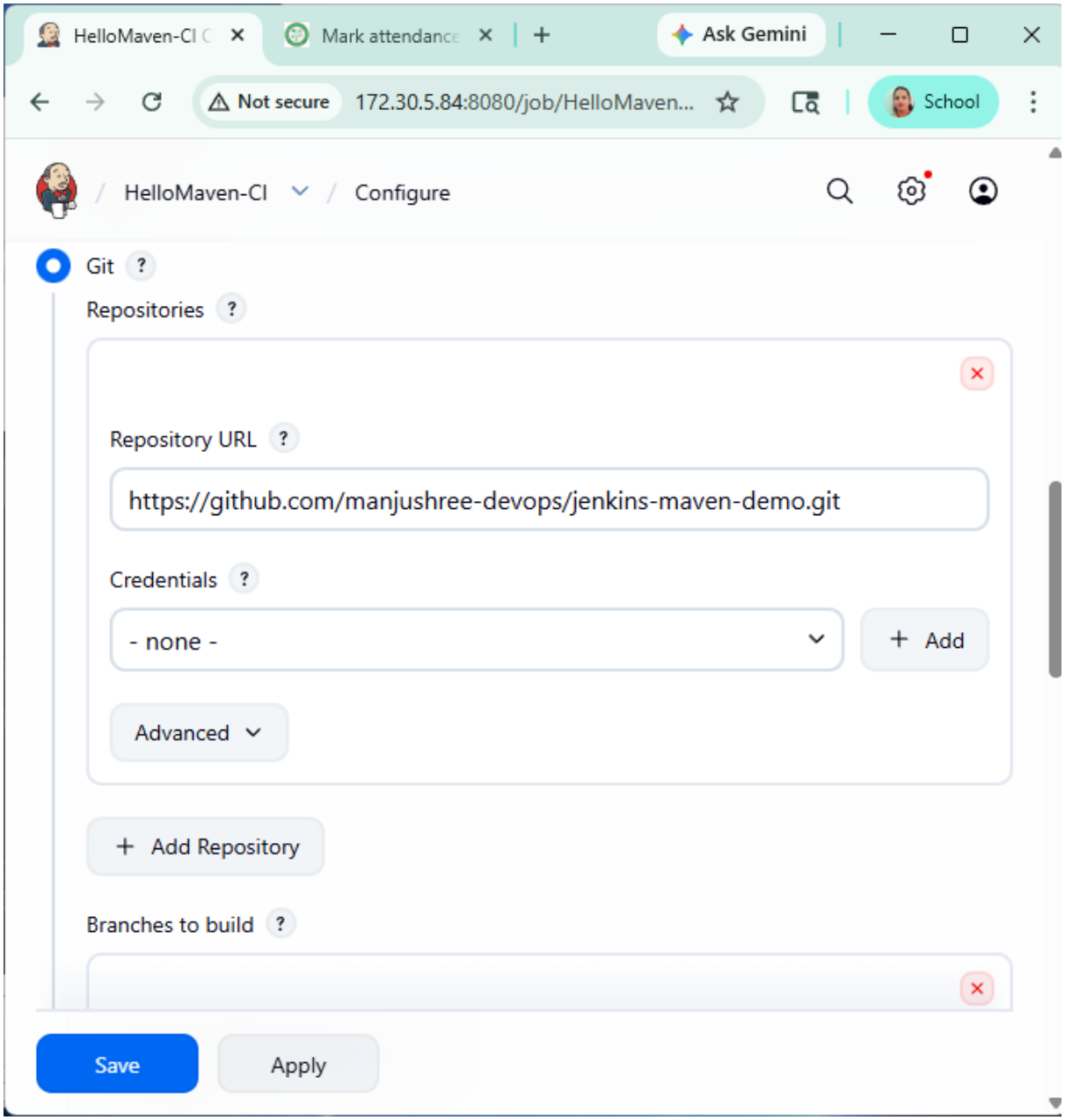
1. Log into Jenkins: Open your browser and navigate to your Jenkins URL (e.g., `http://localhost:8080`).
2. Create a New Job:
    - Click "New Item" on the Jenkins dashboard.
    - Enter Item Name: e.g., `HelloMaven-CI`. Don't change the project name (if a project with this name already exists then delete the existing project),
    - Select "Freestyle project" and click "OK".



2. Configure Source Code Management (SCM)

Scroll to the "Source Code Management" Section:

- Select "Git".
- Repository URL: Enter your repository URL: https://github.com/manjushree-devops/jenkins-maven-demo.git
- Credentials: If the repository is private, click "Add" and provide the necessary credentials. (Not needed for the given public repository).
- Branch Specifier: (e.g., `*/main` or `*/master`). Use `*/main` if you are using above link.



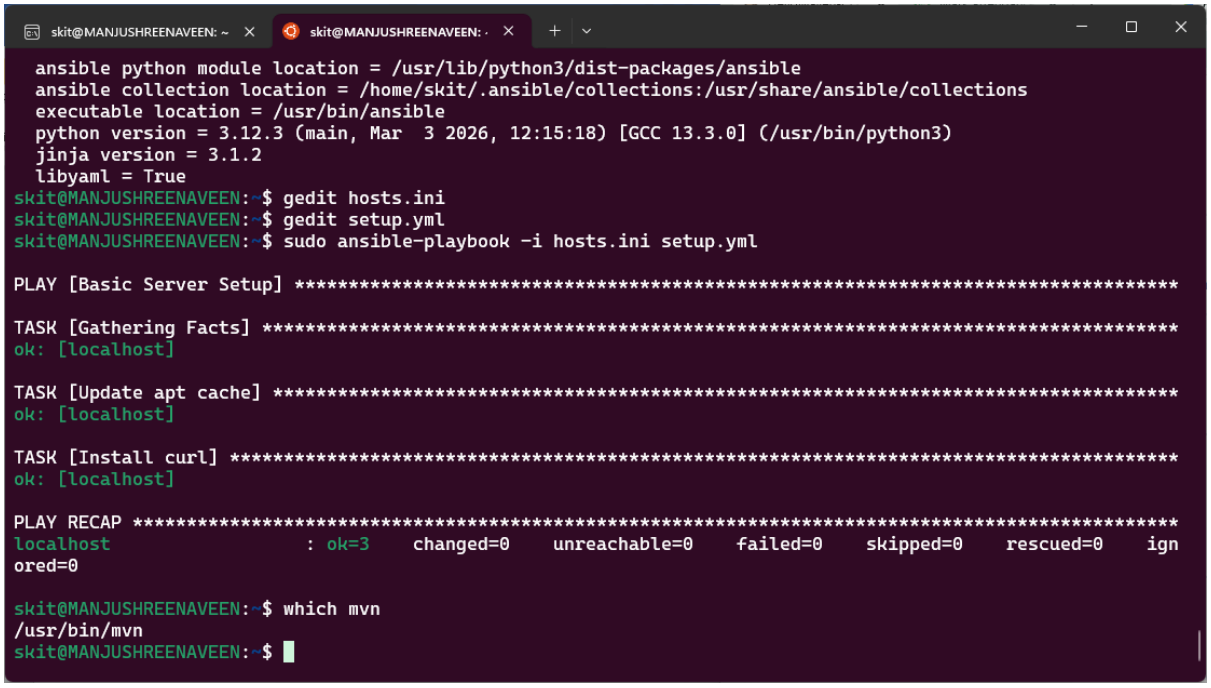
3. Add Build Steps

1. Scroll Down to the "Build" Section.
2. Add the Maven Build Step: Click "Add build step" and select **"Execute shell".**

   Configure the Shell Command:

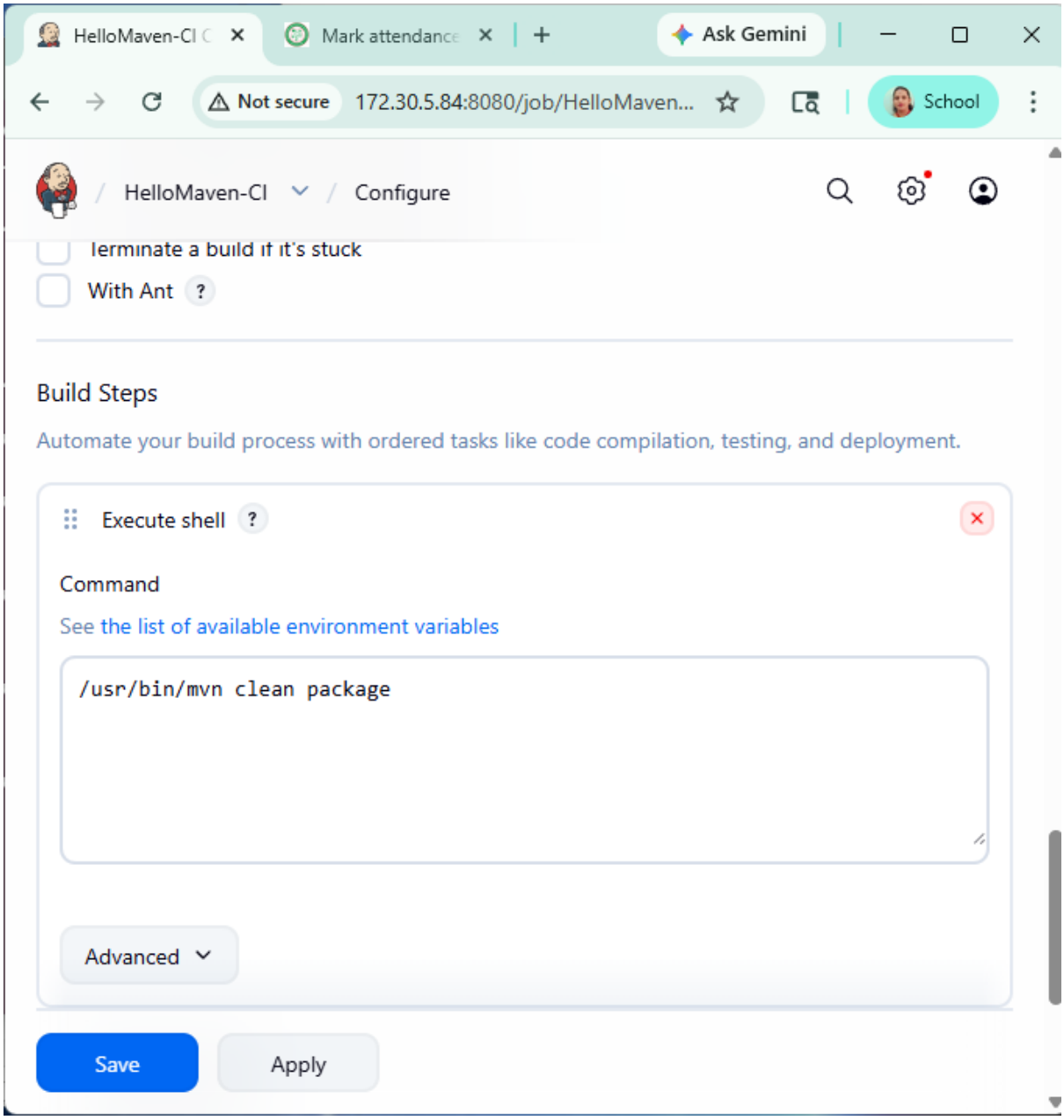
   To find the full path to your `mvn` executable on open terminal in Ubuntu and run:

   ```
   which mvn
   ```



   The output will give you the full path (e.g., `/usr/bin/mvn` or `/opt/maven/bin/mvn`). Use this path in the command below in Jenkins Build Action Shell.

   ```
   #/path/to/your/maven/bin/mvn clean package
   # Example:
   /usr/bin/mvn clean package
   ```



3. Add the Ansible Deployment Step: Click "Add build step" again and select "Execute shell" (or "Execute Windows batch command" on Windows).

   Configure the Shell Command: In the command box, add a command to trigger your Ansible playbook.

   Note: It is highly recommended to upload your `hosts.ini` and `deploy.yml` files to your GitHub repository alongside your project code. This simplifies referencing them in Jenkins and keeps your automation code version-controlled.

   If your files are in the root of your repository (follow this command if you are using given github link), the command would be:

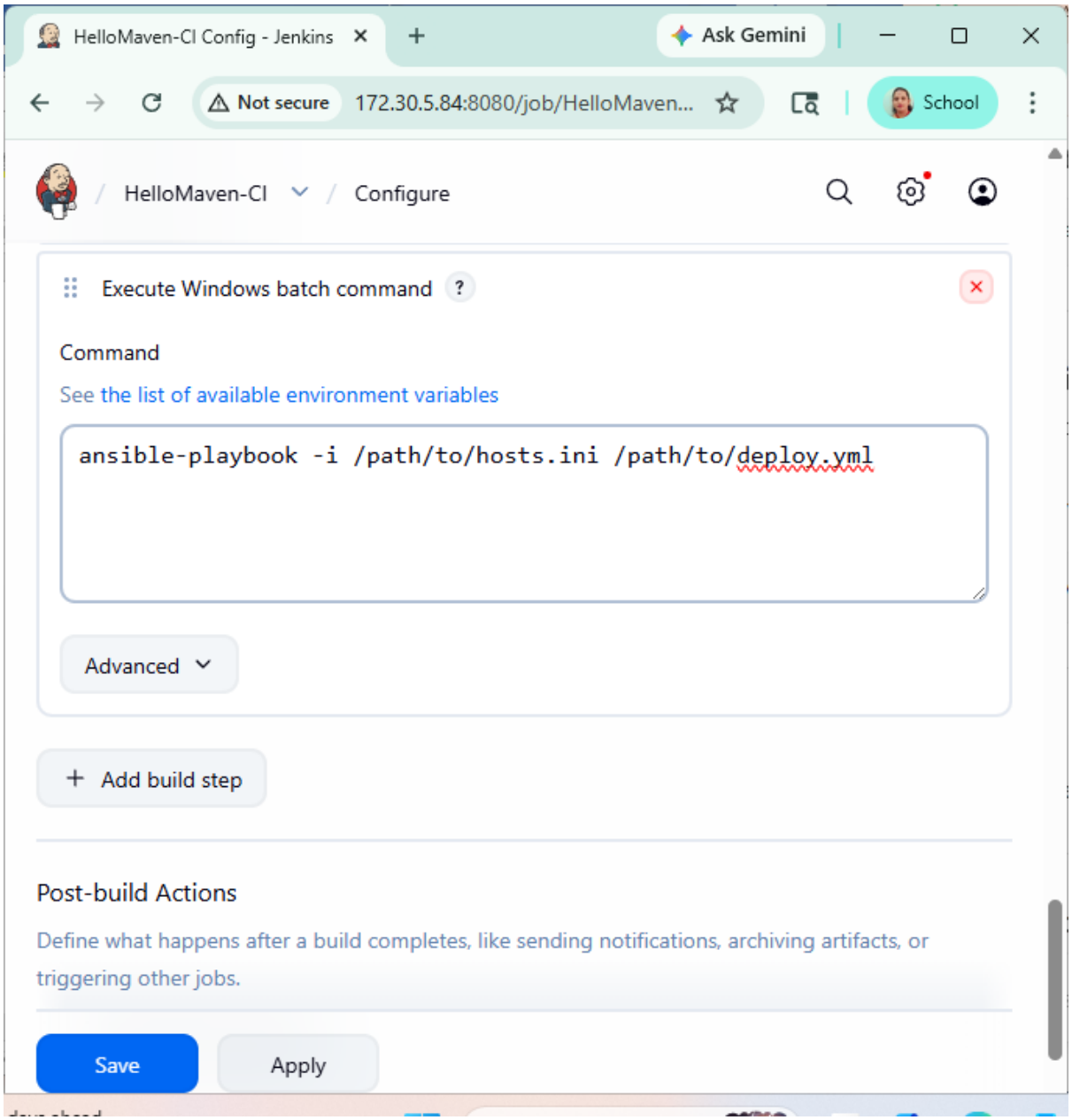
```
ansible-playbook -i hosts.ini deploy.yml
```

If you are not storing them in GitHub, use the full paths:

```
ansible-playbook -i /path/to/hosts.ini /path/to/deploy.yml
```

Note:

- Replace `/path/to/hosts.ini` with the full path to your Ansible inventory file if not in GitHub.
- Replace `/path/to/deploy.yml` with the full path to your Ansible deployment playbook if not in GitHub.
- This command will run after the successful Maven build step.



4. Archiving the Artifact

After the Maven build completes, you need to archive the generated artifact so that it can be used later by the deployment process.

1. Scroll Down to the "Post-build Actions" Section:

- Click "Add post-build action" and select "Archive the artifacts".

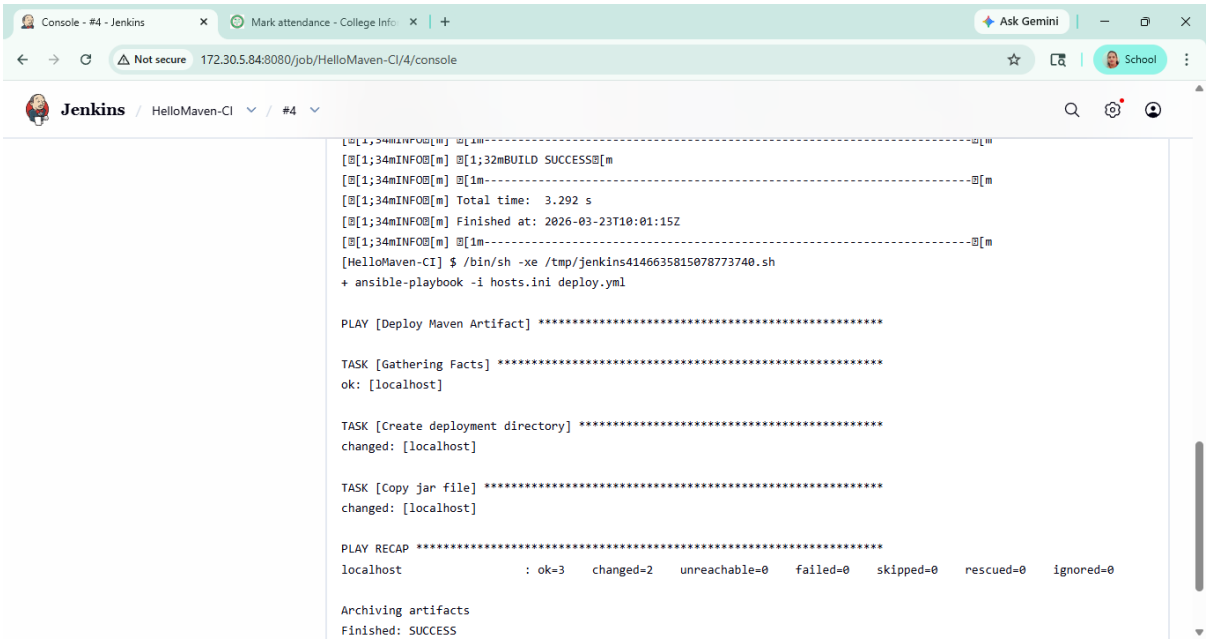
2. Configure Artifact Archiving:

- Files to Archive: Type: `target/*.jar`
- This pattern tells Jenkins to archive any JAR file found in the `target` directory.

---

5. Save the Jenkins Job:

- Click "Save" at the bottom of the configuration page.



---

6. Writing an Ansible Playbook for Deployment (This step is should be skipped if you are using provided github link)

Create an Ansible playbook that deploys the Maven artifact (the JAR file) generated by Jenkins to a target directory. This version does not require `sudo`.

Note: If you are using the provided GitHub repository (`https://github.com/devops-ds/your-maven-project.git`), the `hosts.ini` and `deploy.yml` files are already included in the repository. You can skip the steps below for creating these files and proceed to step 7.

A. Create an Inventory File

If you haven't already, create an inventory file (e.g., `hosts.ini`) that targets the deployment machine. For a local deployment, use:

```
[local]
localhost ansible_connection=local
```

Note: It is highly recommended to upload this `hosts.ini` file to your GitHub repository alongside your project code for easier management and version control.

B. Create the Deployment Playbook

1. Open Your Text Editor and create a file called `deploy.yml`:

```
gedit deploy.yml
```

1. Enter the Following YAML Content:

```
---
- name: Deploy Maven Artifact (Jenkins User Accessible)
  hosts: local # Or the target host/group accessible by Jenkins user
  # become: yes # Removed to avoid requiring sudo

  tasks:
    - name: Ensure deployment directory exists
      ansible.builtin.file:
        path: "/var/lib/jenkins/deployment" # Example path accessible by
Jenkins user
        state: directory
        mode: '0755' # Standard permissions

    - name: Copy the artifact to the deployment directory
      ansible.builtin.copy:
        # Corrected source path to match the actual artifact name from the
Maven build
        src: "/var/lib/jenkins/workspace/HelloMaven-CI/target/maven-junit5-
example-1.0-SNAPSHOT.jar"
        dest: "/var/lib/jenkins/deployment/HelloMaven.jar" # Example path
accessible by Jenkins user
        remote_src: yes # Indicates the source file is on the remote host
(Jenkins agent)
```

1. Save and Exit the File.

---

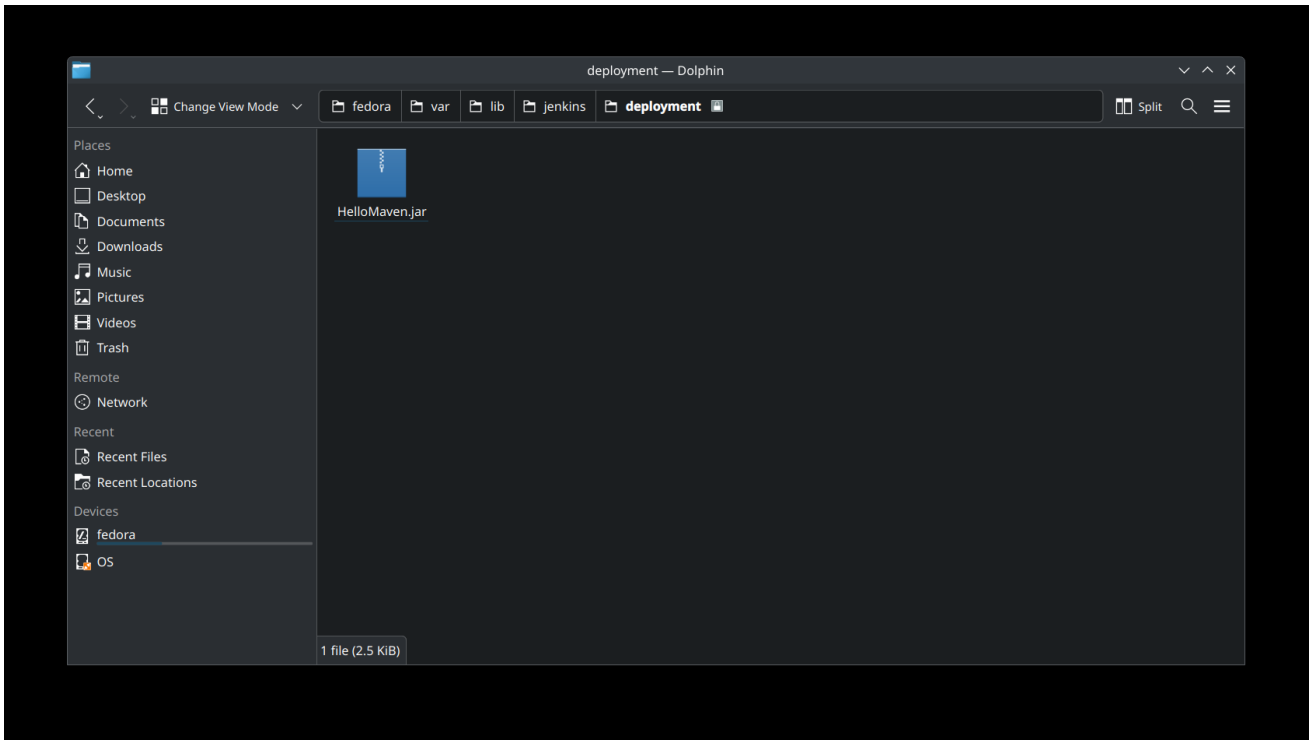
7. Testing the Complete Pipeline

1. Trigger a Build in Jenkins:

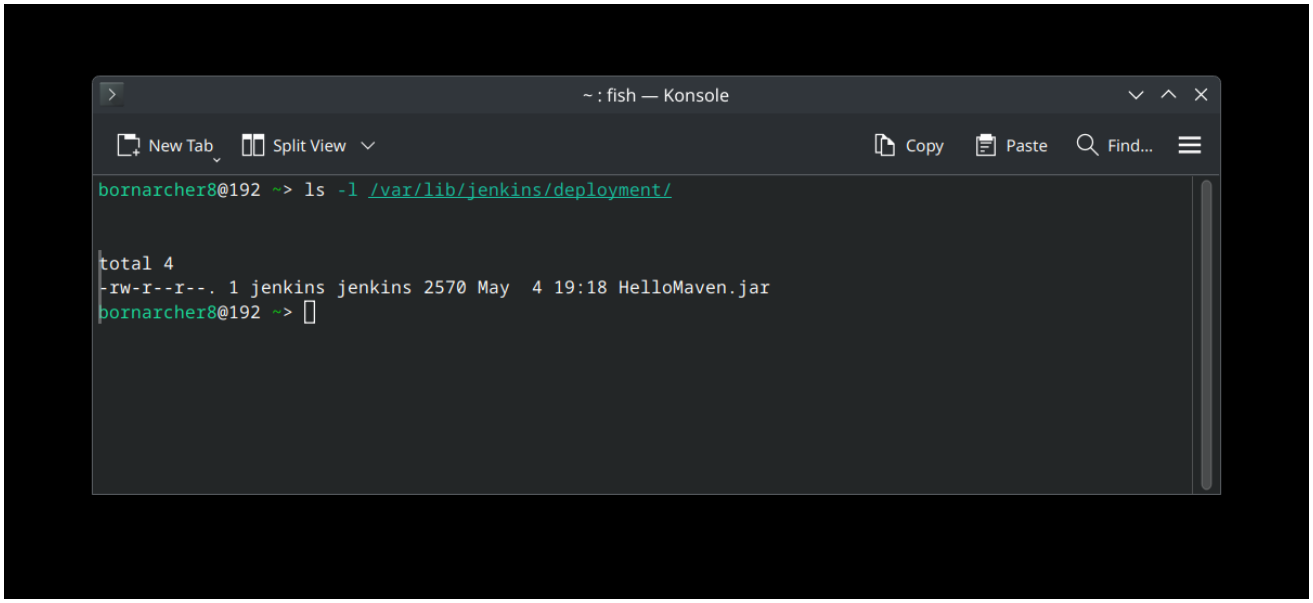
- Navigate to your Jenkins job (`HelloMaven-CI`) and click "Build Now".
- Monitor the build history. Once the build completes, click the build number (e.g., #1) and check the Console Output.
- Look for messages indicating that:
  - The Maven build ran successfully.
  - The shell command executed the Ansible playbook.
  - The artifact was archived.

2. Verify Deployment:

- Open file explore and verify that the artifact has been copied to the destination directory (e.g., `/var/lib/jenkins/deployment/`).



- Check in terminal, run the below comand in terminal:
- `ls -l /var/lib/jenkins/deployment/`
- The output should list the deployed JAR file.


~ : fish — Konsole
New Tab
Split View
Copy
Paste
Find...
bornarcher8@192 ~> ls -l /var/lib/jenkins/deployment/
total 4
-rw-r--r--. 1 jenkins jenkins 2570 May 4 19:18 HelloMaven.jar
bornarcher8@192 ~>